

Prova Técnica - Desenvolvedor Backend Python/FastAPI

Prazo de entrega: 7 dias corridos a partir do recebimento

Tempo estimado: 6-8 horas

O que será valorizado:

- **Raciocínio técnico** - justificar decisões arquiteturais
- **Experiência prática** - demonstrar vivência com problemas reais
- **Pragmatismo** - soluções simples e efetivas
- **Documentação clara** - cliente não-técnico precisa entender
- **Tratamento de erros** - scripts robustos, não apenas "happy path"

Observações Importantes:

- Esta prova foi desenhada para um ambiente AI-First. O uso de ferramentas de IA é permitido e incentivado. Esperamos, porém, que todas as decisões técnicas, validações e entregas finais reflitam seu próprio julgamento de engenharia.
- Comente decisões não-óbvias no código
- Se travar, **documente o que tentou** - isso vale pontos

O que NÃO precisa fazer:

- Keycloak real configurado (pode mockar JWT)
- CI/CD completo (descrever é suficiente)
- Frontend
- Busca otimizada (básico funcional é ok)

Entrega

Repositório **privado** GitHub/GitLab, adicionando guilherme.rabelo@snapforensics.com como colaborador. Ao finalizar enviar um email com o assunto Prova Técnica – [Seu Nome] para o mesmo email acima além do profissional de RH que está acompanhando o seu processo.

Contexto do Projeto

Você trabalha em uma empresa que desenvolve uma **plataforma de inteligência investigativa** distribuída on-premises para órgãos de controle, compliance corporativo e investigações financeiras.

A plataforma é composta por **3 aplicações principais**:

1. **Analytics** - Geração de relatórios analíticos, dashboards e exportações
2. **Investigator** - Visualização de grafos de relacionamento, linha do tempo e análise de vínculos
3. **Case Manager** - Gestão de casos investigativos, atribuição de tarefas e workflow

PARTE 1 — Arquitetura de Autenticação Multi-Aplicação

Cenário:

A empresa quer uma solução de SSO onde:

- Usuário faz login **uma vez** e acessa todas apps permitidas
- Cada app controla **suas próprias permissões** independentemente
- Remover acesso de uma app **não afeta** as outras
- Auditoria: saber **qual app** o usuário acessou e **quando**

A equipe tem uma decisão a tomar:

Decisão Arquitetural: Realm Único ou Múltiplos?

Você tem **duas opções**:

Opção A: Realm único com múltiplos clients

Keycloak Realm: "plataforma"

└─ Client: analytics-api

└─ Client: investigator-api

└─ Client: case-manager-api

Opção B: Múltiplos realms

Keycloak

└─ Realm: analytics

└─ Realm: investigator

└─ Realm: case-manager

Questões:

1. Decisão Fundamentada (escolha A ou B)

Explique:

- Qual opção você escolheu e **por quê?**
- Trade-offs de cada abordagem (complexidade vs isolamento vs UX)
- Como implementar SSO na sua escolha?
- Como gerenciar permissões independentes por app?

2. Modelo de Permissionamento

Desenhe (pode ser texto/diagrama) como funcionaria:

Exemplo de usuário:

João Silva

└─ Analytics: role "viewer" → pode ver relatórios associados ao João

└─ Investigator: role "senior-investigator" → acesso total

└─ Case Manager: SEM ACESSO

Responda:

- Como armazenar essas permissões?
- Como a API valida se usuário pode acessar um endpoint X?
- Como fazer para que token JWT carregue apenas permissões relevantes?
- Como implementar auditoria de acessos por aplicação?

3. Análise de Cenários

Como sua arquitetura resolveria:

Cenário 1: Admin precisa dar acesso ao Analytics para 50 usuários novos, mas já existem 300 usuários com acesso ao Investigator. Como fazer sem afetar os existentes?

Cenário 2: Investigator precisa chamar endpoint do Analytics internamente (API-to-API). Como autenticar? Service account? Mesmo JWT do usuário?

PARTE 2 — Implementação endpoint Multi-Aplicação

Cenário:

Você precisa implementar um endpoint de **busca unificada** que é compartilhado pelas 3 aplicações (Analytics, Investigator, Case Manager), mas que se comporta de forma diferente dependendo de **qual aplicação** e **quais permissões** o usuário possui.

Desafio: O mesmo endpoint `/api/v1/search` precisa:

- Saber de qual aplicação veio a requisição
- Validar se o usuário tem permissão para aquela aplicação específica
- Buscar apenas nos dados relevantes para aquela aplicação
- Retornar resultados formatados apropriadamente
- Registrar a busca para auditoria

Tabelas a serem consideradas:

- analytics_reports (id, title, content, created_at)

- investigator_entities (id, type, name, data, created_at)
- case_manager_cases (id, title, assigned_to, status, created_at)
- search_audit_log (id, user_id, app, query, timestamp)

Regras para pesquisa:

Analytics: Permissão: **analytics:search**

- Busca apenas em: conteúdo dos relatórios gerados
- Retorna dados agregados (sem detalhes sensíveis)

Investigator: Permissão: **investigator:search**

- Busca entidades dos tipos: pessoas, empresas, transações, documentos
- Retorna dados completos

Case Manager: Permissão: **case-manager:search**

- Busca apenas: casos atribuídos ao usuário
- Retorna apenas metadados (não busca conteúdo)

O que você deve entregar:

1. Modelos das tabelas criadas

2. Migration Alembic contendo as tabelas acima, indexes necessários, dados de seed (10 registros para as tres primeiras tabelas)

3. Endpoint Implementado com rotas FastAPI exigindo as permissões necessárias, logica de busca contextual por aplicativo, agregacao de resultado (em caso de multiplas permissões) e o uso da auditoria

4. Testes (pytest)

Pelo menos 5 testes:

- Usuário com analytics:search busca e recebe apenas dados do Analytics
- Usuário com investigator:search busca e recebe dados do Investigator

- Usuário com ambas permissões recebe resultados agregados
- Usuário sem permissão recebe HTTP 403
- Busca é registrada no audit log

5. Autenticação JWT

- Implementar dependencia
- Extrair: user_id, app_client_id, permissions
- Pode mockar Keycloak (biblioteca python-jose ou outra!)
- Documentar estrutura esperada do JWT

PARTE 3 — Incidente em Produção

Cenário:

Sexta-feira, 17:30. Uma nova versão da plataforma foi implantada em produção. Minutos após o deploy, a equipe começa a receber alertas e chamados de usuários.

Sintomas observados:

- Latência média da API aumentou de 200ms para mais de 4 segundos
- Utilização de CPU do PostgreSQL atingiu 95%
- Diversas buscas estão retornando timeout
- Usuários relatam lentidão generalizada na plataforma

O deploy continha as seguintes alterações:

- Novo endpoint de busca
- Nova funcionalidade de auditoria
- Nova migration de banco de dados adicionando índices

Questões

Explique como você conduziria a investigação e resolução do incidente.

Aborde pelo menos os seguintes pontos:

- Quais seriam suas ações imediatas?
- Quais hipóteses levantaria inicialmente?
- Como identificaria a causa raiz?
- Você realizaria rollback imediatamente? Por quê?
- Quais métricas, logs ou ferramentas utilizaria para apoiar a investigação?
- Como comunicaria o status do incidente para as partes interessadas?
- Após a resolução, quais ações preventivas adotaria para evitar recorrência?

PARTE 4 — Revisão de Trade-offs de Engenharia

Cenário:

A implementação foi concluída e está pronta para entrar em produção.

Durante o planejamento da próxima release, o Product Owner informa que o time possui capacidade para implementar apenas **3 melhorias** dentre as listadas abaixo.

Possíveis melhorias:

- Paginação dos resultados de busca
- Rate limiting
- Cache utilizando Redis
- Instrumentação e observabilidade (OpenTelemetry)
- Ampliação da cobertura de testes automatizados
- Pipeline de CI com validações automáticas
- Modelo de permissões mais granular
- Busca textual avançada (full text search)
- Relevância/ranqueamento dos resultados
- Estratégia aprimorada de rollback de deploy

Questões

Responda:

- Quais 3 melhorias você priorizaria?

- Em qual ordem elas seriam executadas?
- Quais critérios utilizou para definir essa priorização?
- Quais riscos você considera aceitáveis assumir neste momento?

PARTE 5 — Relatório de Uso de IA

Cenário:

Nossa empresa adota um fluxo de desenvolvimento AI-First. O uso de ferramentas baseadas em LLM é permitido e incentivado durante a realização desta prova.

Não avaliamos a ausência de uso de IA. Avaliamos a capacidade de utilizá-la de forma produtiva, crítica e responsável.

O que deve ser entregue

Crie um arquivo chamado `AI_ENGINEERING_LOG.md` contendo:

1. Ferramentas utilizadas

Liste as ferramentas utilizadas durante a prova.

2. Casos em que a IA ajudou

Descreva até 3 situações em que o uso de IA acelerou ou facilitou seu trabalho.

Explique brevemente:

- Qual era o problema
- Como a IA ajudou
- Qual foi o resultado

3. Casos em que a IA estava errada

Descreva pelo menos 1 situação em que uma sugestão da IA foi incorreta, incompleta ou inadequada para o contexto

Explique:

- O que foi sugerido
- Por que a sugestão era problemática
- Como você identificou o problema
- O que decidiu fazer em vez disso

4. Decisões de Engenharia

Descreva pelo menos 2 decisões importantes tomadas por você durante a implementação.

Explique:

- Quais alternativas foram consideradas
- Qual decisão foi tomada
- Por que ela foi escolhida

O objetivo desta seção é compreender seu processo de trabalho, sua capacidade crítica e a forma como utiliza ferramentas de IA para apoiar decisões técnicas.

5. Exemplos de Interações com IA (Opcional)

Opcionalmente, compartilhe até 3 interações com ferramentas de IA que você considera relevantes durante o desenvolvimento da solução.